ICPC NOTEBOOK 2025

Tabla de contenido

Bloque 1: utilidades base y estructuras esenciales 4
Template base (Fast I/O + Macros + Setup ICPC) 4
Estructuras comúnes y funciones básicas 5
Modular arithmetic 5
Prefix sum y difference array 6
Binary search / lower_bound / upper_bound 6
Custom sort avanzado 7
Bloque 2: Algoritmos clásicos de grafos y búsqueda 7
BFS (Breadth-First Search) 7
DFS (Depth-First Search) 8
Floyd-Warshall 8
BFS 0-1 (para grafos con pesos 0 o 1) 8
Dijkstra 9
Union-Find (Disjoint Set Union) 10
Kruskal (Minimum Spanning Tree) 10
Topological Sort (Kahn + DFS) 11
Binary Exponentiation (potencia rápida) 11
Kadane (máximo subarreglo) 12
Two pointers / Sliding window 12
Bloque 3: Estructuras avanzadas 12
Fenwick Tree (BIT, suma prefijo y suma de rango) 12
Segment Tree (range add + range sum, lazy propagation) 13
Sparse Table (RMQ estático min) 14
Trie de palabras (a–z) 14
Heap / Priority Queue (trucos básicos) 15
Criba de Eratóstenes + SPF (mínimo factor primo) 15
GCD/LCM + Euclides Extendido + Inverso modular 16
Chinese Remainder Theorem (CRT) para módulos coprimos 16
Z-Function (matching de prefijos) 16
Exponenciación de matrices (ej. Fibonacci, genérico) 17
KMP (prefix-function + encontrar patrones) 17
Rolling Hash doble (búsqueda de substrings) 18

Kosaraju (SCC) 18
Suffix Array + LCP (Kasai) 19
Tarjan: Puentes y puntos de articulación 20
Kuhn (Matching bipartito máximo) 20
Dinic (Flujo Máximo) 21
Bloque 4: Programación dinámica y geometría **21**
Bitmask DP (TSP básico) 22
Digit DP (conteo de números <= N con restricción) 22
Geometría: Punto, orientación, intersección de segmentos 23
DP sobre árboles (tamaños de subárbol, ejemplo) 23
Convex Hull (Monotone Chain) + Área de polígono 24
Ternary Search (unimodal en reales) 24
Binary Search en la respuesta (patrón) 24
Utilidades de bits 25
Mochila completa (unbounded knapsack) 25
LIS y LDS O(n log n) 25
Bitmask DP 26
Digit DP 26
DP sobre árboles 27
DP con divide & conquer optimization 28
Cross product y orientación 28
Intersección de segmentos 28
Estructuras Point, Line, Segment 29
Polygon area (Shoelace) 30

Bloque 1: utilidades base y estructuras esenciales

Template base (Fast I/O + Macros + Setup ICPC)

```
#include <bits/stdc++.h>
using namespace std;
// --- Definiciones Rápidas ---
#define ll long long
#define all(x) (x).begin(), (x).end()
#define pb push_back
#define fi first
#define se second
#define endl '\n'

// --- Constantes (activar si se usan) ---
// const int MOD = 1e9+7;
// const ll INF = 1e18;

// --- Función Principal del Problema ---
void solve() {
    // El código va aquí

}

// --- Main ---
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    int t = 1;
    // cin >> t; // Descomentar para múltiples casos de prueba
    while (t--) {
        solve();
    }
    return 0;
}
```

Estructuras comúnes y funciones básicas

```
// Estructura de impresión y lectura genérica
template<typename T>
void print_vector(const vector<T>& v, string sep=" "){
    for(int i=0;i<(int)v.size();i++){
        if(i) cout<<sep;
        cout<<v[i];
    }
    cout<<'\n';
}

template<typename T>
void read_vector(vector<T>& v, int n){
    v.resize(n);
    for(auto &x:v) cin>>x;
}

// Comparador personalizado (ejemplo para ordenar pares)
bool cmp_pair(const pair<int,int>& a, const pair<int,int>& b){
    if(a.fi==b.fi) return a.se<b.se;
    return a.fi<b.fi;
}
```

Modular arithmetic

```
// Potencia modular (a^b mod m)
ll modpow(ll a, ll b, ll m=MOD){
    ll res=1; a%=m;
    while(b>0){
        if(b&1) res=res*a%m;
        a=a*a%m;
        b>>=1;
    }
    return res;
}

// Inverso modular (a^-1 mod m) con Fermat (m primo)
ll modinv(ll a, ll m=MOD){ return modpow(a,m-2,m); }

// nCr % MOD usando factoriales precalculados
const int MAXN = 1e6+5;
ll fact[MAXN], invfact[MAXN];

void init_fact(){
    fact[0]=1;
    for(int i=1;i<MAXN;i++) fact[i]=fact[i-1]*i%MOD;
    invfact[MAXN-1]=modinv(fact[MAXN-1]);
    for(int i=MAXN-2;i>=0;i--) invfact[i]=invfact[i+1]*(i+1)%MOD;
}

ll nCr(int n,int r){
    if(r<0||r>n) return 0;
    return fact[n]*invfact[r]%MOD*invfact[n-r]%MOD;
}
```

Prefix sum y difference array

```
// Prefix sum para consultas de rango
vector<ll> build_prefix(const vector<ll>& a){
    int n=a.size();
    vector<ll> pref(n+1,0);
    for(int i=0;i<n;i++) pref[i+1]=pref[i]+a[i];
    return pref;
}
ll range_sum(const vector<ll>& pref, int l, int r){
    // suma en [l, r)
    return pref[r]-pref[l];
}
// Difference array para actualizaciones por rango
void apply_diff(vector<ll>& diff, int l, int r, ll val){
    diff[l]+=val;
    if(r<(int)diff.size()) diff[r]-=val;
}
vector<ll> build_from_diff(const vector<ll>& diff){
    vector<ll> a(diff.size());
    a[0]=diff[0];
    for(int i=1;i<(int)diff.size();i++) a[i]=a[i-1]+diff[i];
    return a;
}
```

Binary search / lower_bound / upper_bound

```
// Ejemplo genérico de búsqueda binaria
int binary_search_example(const vector<int>& a, int x){
    int l=0, r=(int)a.size()-1, ans=-1;
    while(l<=r){
        int m=(l+r)/2;
        if(a[m]<=x){
            ans=m;
            l=m+1;
        }else r=m-1;
    }
    return ans;
}

// Uso de lower_bound y upper_bound
void example_bounds(){
    vector<int> v={1,3,3,5,7};
    int x=3;
    int lb = lower_bound(all(v), x)-v.begin(); // primer >= x
    int ub = upper_bound(all(v), x)-v.begin(); // primer > x
    // ocurrencias de x = ub - lb
}
```

Custom sort avanzado

```
struct Person{
    string name;
    int age, score;
};

// Ordenar por score descendente, luego edad ascendente
bool cmp_person(const Person &a, const Person &b){
    if(a.score==b.score) return a.age<b.age;
    return a.score>b.score;
}

void example_sort(){
    vector<Person> v={{"Ana",20,90},{"Luis",21,95},{"Ana",19,95}};
    sort(all(v), cmp_person);
}
```

Bloque 2: Algoritmos clásicos de grafos y búsqueda

BFS (Breadth-First Search)

```
// BFS en grafo no ponderado
vector<int> bfs(const vector<vector<int>>& g, int s){
    int n = g.size();
    vector<int> dist(n, -1);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : g[u]){
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist; // distancias desde s
}
```

DFS (Depth-First Search)

```
// DFS recursivo
void dfs_rec(int u, const vector<vector<int>>& g, vector<int>& vis){
    vis[u] = 1;
    for(int v : g[u]){
        if(!vis[v]) dfs_rec(v, g, vis);
    }
}

// DFS iterativo
void dfs_it(int s, const vector<vector<int>>& g){
    int n = g.size();
    vector<int> vis(n, 0);
    stack<int> st;
    st.push(s);
    while(!st.empty()){
        int u = st.top(); st.pop();
        if(vis[u]) continue;
        vis[u] = 1;
        for(int v : g[u]) if(!vis[v]) st.push(v);
    }
}
```

Floyd-Warshall

```
// Complejidad O(n^3)
vector<vector<ll>> floyd(vector<vector<ll>> dist){
    int n = dist.size();
    for(int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(dist[i][k]<INF && dist[k][j]<INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
    return dist;
}
```

BFS 0-1 (para grafos con pesos 0 o 1)

```
vector<ll> bfs01(int n, vector<vector<pair<int,int>>>& g, int s){
    deque<int> dq;
    vector<ll> dist(n, INF);
    dist[s] = 0;
    dq.push_front(s);
    while(!dq.empty()){
        int u = dq.front(); dq.pop_front();
        for(auto [v, w] : g[u]){
            if(dist[v] > dist[u] + w){
                dist[v] = dist[u] + w;
                if(w == 1) dq.push_back(v);
                else dq.push_front(v);
            }
        }
    }
    return dist;
}
```

Dijkstra

```
struct Edge { int to; ll w; };

vector<ll> dijkstra(int n, vector<vector<Edge>>& g, int s){
    vector<ll> dist(n, INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    dist[s] = 0;
    pq.push({0,s});
    while(!pq.empty()){
        auto [d,u] = pq.top(); pq.pop();
        if(d != dist[u]) continue;
        for(auto &e : g[u]){
            if(dist[e.to] > d + e.w){
                dist[e.to] = d + e.w;
                pq.push({dist[e.to], e.to});
            }
        }
    }
    return dist;
}
```

Bellman-Ford

```
struct EdgeBF { int u, v; ll w; };

vector<ll> bellman_ford(int n, vector<EdgeBF>& edges, int s){
    vector<ll> dist(n, INF);
    dist[s] = 0;
    for(int i=0;i<n-1;i++){
        for(auto &e: edges){
            if(dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w){
                dist[e.v] = dist[e.u] + e.w;
            }
        }
    }
    // detectar ciclo negativo
    for(auto &e: edges){
        if(dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w){
            cerr << "Ciclo negativo detectado\n";
            break;
        }
    }
    return dist;
}
```

Union-Find (Disjoint Set Union)

```
struct DSU{
    vector<int> p, sz;
    DSU(int n){ p.resize(n); sz.assign(n,1); iota(p.begin(), p.end(), 0); }

    int find(int x){ return p[x]==x?x:p[x]=find(p[x]); }

    bool unite(int a, int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(sz[a]<sz[b]) swap(a,b);
        p[b]=a;
        sz[a]+=sz[b];
        return true;
    }
};
```

Kruskal (Minimum Spanning Tree)

```
struct EdgeK { int u,v; ll w; };

ll kruskal(int n, vector<EdgeK>& edges){
    sort(edges.begin(), edges.end(), [](auto &a, auto &b){ return a.w < b.w; });
    DSU dsu(n);
    ll total=0;
    for(auto &e : edges){
        if(dsu.unite(e.u, e.v)) total += e.w;
    }
    return total;
}

Prim (MST alternativo)
ll prim(int n, vector<vector<Edge>>& g){
    vector<ll> dist(n, INF);
    vector<int> vis(n, 0);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    dist[0]=0; pq.push({0,0});
    ll total=0;
    while(!pq.empty()){
        auto [d,u]=pq.top(); pq.pop();
        if(vis[u]) continue;
        vis[u]=1;
        total += d;
        for(auto &e: g[u]){
            if(!vis[e.to] && e.w < dist[e.to]){
                dist[e.to]=e.w;
                pq.push({e.w, e.to});
            }
        }
    }
    return total;
}
```

Topological Sort (Kahn + DFS)

```
// Versión con Kahn (BFS)
vector<int> topo_kahn(int n, const vector<vector<int>>& g){
    vector<int> indeg(n,0);
    for(int u=0;u<n;u++)
        for(int v:g[u]) indeg[v]++;
    queue<int> q;
    for(int i=0;i<n;i++) if(indeg[i]==0) q.push(i);
    vector<int> order;
    while(!q.empty()){
        int u=q.front(); q.pop();
        order.push_back(u);
        for(int v:g[u]){
            if(--indeg[v]==0) q.push(v);
        }
    }
    return order; // vacío si hay ciclo
}

// Versión con DFS
void topo_dfs_util(int u, vector<int>& vis, vector<int>& order, const vector<vector<int>>& g){
    vis[u]=1;
    for(int v:g[u]) if(!vis[v]) topo_dfs_util(v, vis, order, g);
    order.push_back(u);
}
vector<int> topo_dfs(int n, const vector<vector<int>>& g){
    vector<int> vis(n,0), order;
    for(int i=0;i<n;i++) if(!vis[i]) topo_dfs_util(i, vis, order, g);
    reverse(order.begin(), order.end());
    return order;
}
```

Binary Exponentiation (potencia rápida)

```
ll binpow(ll a, ll b){
    ll res=1;
    while(b>0){
        if(b&1) res=res*a;
        a=a*a;
        b>>=1;
    }
    return res;
}
```

Kadane (máximo subarreglo)

```
ll kadane(const vector<ll>& a){
    ll best = -INF, curr = 0;
    for(ll x : a){
        curr = max(x, curr+x);
        best = max(best, curr);
    }
    return best;
}
```

Two pointers / Sliding window

```
int longest_subarray_sum_leq_k(const vector<int>& a, int k){
    int n=a.size();
    int l=0, sum=0, best=0;
    for(int r=0;r<n;r++){
        sum += a[r];
        while(sum > k) sum -= a[l++];
        best = max(best, r-l+1);
    }
    return best;
}
```

Bloque 3: Estructuras avanzadas

Fenwick Tree (BIT, suma prefijo y suma de rango)

```
// Fenwick Tree 1-indexed. Soporta add(pos, val) y sum(prefijo).
struct Fenwick {
    int n; vector<long long> bit;
    Fenwick(int n=0){init(n);}
    void init(int _n){ n=_n; bit.assign(n+1,0); }
    void add(int idx,long long val){ for(; idx<=n; idx+=idx&-idx)
bit[idx]+=val; }
    long long sum(int idx){ long long r=0; for(; idx>0; idx-=idx&-idx)
r+=bit[idx]; return r; }
    long long range_sum(int l,int r){ return sum(r)-sum(l-1); } // [l,r]
};
```

Segment Tree (range add + range sum, lazy propagation)

```
// Segment Tree con lazy propagation para actualizaciones por rango (add) y consultas de suma.
// Complejidad: build O(n), update/query O(log n).
struct SegTree {
    int n;
    vector<long long> t, lz;
    SegTree(int n=0){init(n);}
    void init(int _n){
        n=1; while(n<_n) n<<=1;
        t.assign(2*n,0);
        lz.assign(2*n,0);
    }
    void build(const vector<long long>& a){
        int m=a.size();
        init(m);
        for(int i=0;i<m;i++) t[n+i]=a[i];
        for(int i=n-1;i>0;i--) t[i]=t[i<<1]+t[i<<1|1];
    }
    void push(int p, int len){
        if(lz[p]==0) return;
        long long v=lz[p];
        t[p<<1]+=v*(len>>1);
        t[p<<1|1]+=v*(len>>1);
        lz[p<<1]+=v; lz[p<<1|1]+=v;
        lz[p]=0;
    }
    void _add(int p,int l,int r,int ql,int qr,long long v){
        if(qr<=l||r<=ql) return;
        if(ql<=l&&r<=qr){
            t[p]+=v*(r-l);
            lz[p]+=v;
            return;
        }
        push(p, r-l);
        int m=(l+r)>>1;
        _add(p<<1,l,m,ql,qr,v);
        _add(p<<1|1,m,r,ql,qr,v);
        t[p]=t[p<<1]+t[p<<1|1];
    }
    void add(int l,int r,long long v){ _add(1,0,n,l,r,v); }
    long long _sum(int p,int l,int r,int ql,int qr){
        if(qr<=l||r<=ql) return 0;
        if(ql<=l&&r<=qr) return t[p];
        push(p, r-l);
        int m=(l+r)>>1;
        return _sum(p<<1,l,m,ql,qr)+_sum(p<<1|1,m,r,ql,qr);
    }
    long long sum(int l,int r){ return _sum(1,0,n,l,r); }
};
```

Sparse Table (RMQ estático min)

```
// Preprocesa O(n log n) y responde RMQ O(1). No soporta actualizaciones.
struct SparseTable {
    int n, K; vector<vector<long long>> st; vector<int> lg;
    void build(const vector<long long>& a){
        n=a.size(); K=32-__builtin_clz(n);
        st.assign(K, vector<long long>(n));
        st[0]=a;
        for(int k=1;k<K;k++){
            for(int i=0;i+(1<<k)<=n;i++){
                st[k][i]=min(st[k-1][i], st[k-1][i+(1<<(k-1))]);
            }
        }
        lg.assign(n+1,0);
        for(int i=2;i<=n;i++) lg[i]=lg[i/2]+1;
    }
    long long query(int l,int r){ // [l,r]
        int k=lg[r-l+1];
        return min(st[k][l], st[k][r-(1<<k)+1]);
    }
};
```

Trie de palabras (a–z)

```
// Trie simple para minúsculas. Soporta insertar y buscar prefijos/palabras.
struct Trie {
    struct Node{ int nxt[26]; int end=0; Node(){ memset(nxt,-1,sizeof(nxt)); } };
    vector<Node> t;
    Trie(){ t.emplace_back(); }
    void insert(const string& s){
        int u=0;
        for(char c:s){
            int x=c-'a';
            if(t[u].nxt[x]==-1){ t[u].nxt[x]=t.size(); t.emplace_back(); }
            u=t[u].nxt[x];
        }
        t[u].end++;
    }
    bool contains(const string& s){
        int u=0;
        for(char c:s){
            int x=c-'a';
            if(t[u].nxt[x]==-1) return false;
            u=t[u].nxt[x];
        }
        return t[u].end>0;
    }
};
```

Heap / Priority Queue (trucos básicos)

```
// K elementos más grandes usando min-heap de tamaño K.
vector<long long> k_largest(const vector<long long>& a, int K){
    priority_queue<long long, vector<long long>, greater<long long>> pq;
    for(long long x:a){
        if((int)pq.size()<K) pq.push(x);
        else if(x>pq.top()){ pq.pop(); pq.push(x); }
    }
    vector<long long> res;
    while(!pq.empty()){ res.push_back(pq.top()); pq.pop(); }
    sort(res.rbegin(), res.rend());
    return res;
}
```

Criba de Eratóstenes + SPF (mínimo factor primo)

```
// Genera primos y SPF hasta N. SPF[i] = menor primo que divide i.
struct Sieve {
    int N; vector<int> primes, spf; vector<bool> is_composite;
    Sieve(int N):N(N),spf(N+1,0),is_composite(N+1,false){
        for(int i=2;i<=N;i++){
            if(!is_composite[i]){ primes.push_back(i); spf[i]=i; }
            for(int p:primes){
                long long x=1LL*p*i;
                if(x>N || p>spf[i]) break;
                is_composite[x]=true; spf[x]=p;
            }
        }
    }
    vector<pair<int,int>> factorize(int x){
        vector<pair<int,int>> f;
        while(x>1){
            int p=spf[x], c=0;
            while(x%p==0){ x/=p; c++; }
            f.push_back({p,c});
        }
        return f;
    }
};
```

GCD/LCM + Euclides Extendido + Inverso modular

```
long long gcdll(long long a,long long b){ return b?gcdll(b,a%b):a; }
long long lcmll(long long a,long long b){ return a/gcdll(a,b)*b; }

long long egcd(long long a,long long b,long long &x,long long &y){
    if(b==0){ x=1; y=0; return a; }
    long long x1,y1; long long g=egcd(b,a%b,x1,y1);
    x=y1; y=x1 - (a/b)*y1; return g;
}
long long mod_inv(long long a,long long m){
    long long x,y; long long g=egcd(a,m,x,y);
    if(g!=1) return -1; x%=m; if(x<0) x+=m; return x;
}
```

Chinese Remainder Theorem (CRT) para módulos coprimos

```
// Resuelve x ≡ r[i] (mod m[i]) con m[i] coprimos. Devuelve (x, M) tal que x mod M es solución.
pair<long long,long long> crt(const vector<long long>& r, const vector<long long>& m){
    long long x=0, M=1;
    for(size_t i=0;i<r.size();i++){
        long long r2=r[i], m2=m[i];
        long long p, q; long long g=egcd(M,m2,p,q);
        if((r2 - x) % g != 0) return {0,-1}; // no solución
        long long tmp = ( (__int128)(r2 - x)/g * p ) % (m2/g);
        x += M * tmp;
        M *= m2/g;
        x = (x%M + M)%M;
    }
    return {x,M};
}
```

Z-Function (matching de prefijos)

```
vector<int> z_function(const string& s){
    int n=s.size(); vector<int> z(n);
    int l=0,r=0;
    for(int i=1;i<n;i++){
        if(i<=r) z[i]=min(r-i+1, z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]-1>r){ l=i; r=i+z[i]-1; }
    }
    return z;
}
```

Exponenciación de matrices (ej. Fibonacci, genérico)

```
struct Mat {
    int n; vector<vector<long long>> a; static const long long MOD=1000000007;
    Mat(int n=0,bool ident=false):n(n),a(n,vector<long long>(n,0)){
        if(ident) for(int i=0;i<n;i++) a[i][i]=1;
    }
    Mat operator*(const Mat& o)const{
        Mat r(n);
        for(int i=0;i<n;i++)
            for(int k=0;k<n;k++) if(a[i][k])
                for(int j=0;j<n;j++)
                    r.a[i][j]=(r.a[i][j]+a[i][k]*o.a[k][j])%MOD;
        return r;
    }
};
Mat mat_pow(Mat base, long long e){
    Mat res(base.n,true);
    while(e){
        if(e&1) res=res*base;
        base=base*base; e>>=1;
    }
    return res;
}
```

KMP (prefix-function + encontrar patrones)

```
vector<int> prefix_function(const string& s){
    int n=s.size(); vector<int> pi(n);
    for(int i=1;i<n;i++){
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) j++;
        pi[i]=j;
    }
    return pi;
}
vector<int> kmp_find(const string& text, const string& pat){
    string s=pat+"#"+text;
    auto pi=prefix_function(s);
    vector<int> pos;
    int m=pat.size();
    for(int i=m+1;i<(int)s.size();i++){
        if(pi[i]==m) pos.push_back(i-2*m);
    }
    return pos;
}
```

Rolling Hash doble (búsqueda de substrings)

```
 struct RH {
    static const long long M1=1000000007, M2=1000000009, B=911382323;
    vector<long long> p1,p2,h1,h2;
    RH(const string& s){
        int n=s.size(); p1.assign(n+1,1); p2.assign(n+1,1);
        h1.assign(n+1,0); h2.assign(n+1,0);
        for(int i=1;i<=n;i++){
            p1[i]=p1[i-1]*B%M1; p2[i]=p2[i-1]*B%M2;
            h1[i]=(h1[i-1]*B + s[i-1])%M1;
            h2[i]=(h2[i-1]*B + s[i-1])%M2;
        }
    }
    pair<long long,long long> get(int l,int r){ // [l,r)
        long long x1=(h1[r]-h1[l]*p1[r-l])%M1; if(x1<0)x1+=M1;
        long long x2=(h2[r]-h2[l]*p2[r-l])%M2; if(x2<0)x2+=M2;
        return {x1,x2};
    }
};
```

Kosaraju (SCC)

```
struct Kosaraju {
    int n; vector<vector<int>> g, gr;
    vector<int> comp, order, used;
    Kosaraju(int n):n(n),g(n),gr(n),comp(n,-1),used(n,0){}
    void add_edge(int u,int v){ g[u].push_back(v); gr[v].push_back(u); }
    void dfs1(int u){ used[u]=1; for(int v:g[u]) if(!used[v]) dfs1(v); order.push_back(u); }
    void dfs2(int u,int c){ comp[u]=c; for(int v:gr[u]) if(comp[v]==-1) dfs2(v,c); }
    int scc(){
        for(int i=0;i<n;i++) if(!used[i]) dfs1(i);
        reverse(order.begin(), order.end());
        int c=0;
        for(int u:order) if(comp[u]==-1) dfs2(u,c++);
        return c;
    }
};
```

Suffix Array + LCP (Kasai)

```
vector<int> suffix_array(string s){
    s.push_back('$');
    int n=s.size();
    vector<int> p(n), c(n);
    vector<pair<char,int>> a(n);
    for(int i=0;i<n;i++) a[i]={s[i],i};
    sort(a.begin(),a.end());
    for(int i=0;i<n;i++) p[i]=a[i].second;
    c[p[0]]=0;
    for(int i=1;i<n;i++) c[p[i]]=c[p[i-1]]+(a[i].first!=a[i-1].first);
    int k=0;
    vector<int> pn(n), cn(n);
    while((1<<k)<n){
        for(int i=0;i<n;i++) p[i]=(p[i] - (1<<k) + n)%n;
        // counting sort by c
        vector<int> cnt(n); for(int x:c) cnt[x]++;
        vector<int> pos(n); pos[0]=0; for(int i=1;i<n;i++) pos[i]=pos[i-1]+cnt[i-1];
        for(int x:p){ int i=c[x]; p[pos[i]++]=x; }
        cn[p[0]]=0;
        for(int i=1;i<n;i++){
            pair<int,int> cur={c[p[i]], c[(p[i]+(1<<k))%n]};
            pair<int,int> prev={c[p[i-1]], c[(p[i-1]+(1<<k))%n]};
            cn[p[i]]=cn[p[i-1]] + (cur!=prev);
        }
        c.swap(cn); k++;
    }
    p.erase(p.begin()); // quitar '$'
    return p;
}
vector<int> lcp_array(const string& s, const vector<int>& sa){
    int n=s.size();
    vector<int> rank(n,0);
    for(int i=0;i<n;i++) rank[sa[i]]=i;
    int k=0; vector<int> lcp(n-1,0);
    for(int i=0;i<n;i++){
        if(rank[i]==n-1){ k=0; continue; }
        int j=sa[rank[i]+1];
        while(i+k<n && j+k<n && s[i+k]==s[j+k]) k++;
        if(rank[i]<n-1) lcp[rank[i]]=k;
        if(k) k--;
    }
    return lcp;
}
```

Tarjan: Puentes y puntos de articulación

```
struct TarjanBridges {
    int n, timer;
    vector<vector<int>> g;
    vector<int> tin, low, is_art;
    vector<pair<int,int>> bridges;
    TarjanBridges(int n):n(n),g(n),tin(n,-1),low(n,-1),is_art(n,0),timer(0){}
    void add_edge(int u,int v){ g[u].push_back(v); g[v].push_back(u); }
    void dfs(int u,int p=-1){
        tin[u]=low[u]=timer++;
        int children=0;
        for(int v:g[u]){
            if(v==p) continue;
            if(tin[v]!=-1){
                low[u]=min(low[u], tin[v]);
            }else{
                dfs(v,u);
                low[u]=min(low[u], low[v]);
                if(low[v]>tin[u]) bridges.push_back({u,v});
                if(low[v]>=tin[u] && p!=-1) is_art[u]=1;
                children++;
            }
        }
        if(p==-1 && children>1) is_art[u]=1;
    }
    void run(){ for(int i=0;i<n;i++) if(tin[i]==-1) dfs(i); }
};
```

Kuhn (Matching bipartito máximo)

```
// Lado izquierdo [0..nL-1], derecho [0..nR-1]
struct Kuhn {
    int nL, nR; vector<vector<int>> g; vector<int> mt; vector<int> vis;
    Kuhn(int nL,int nR):nL(nL),nR(nR),g(nL),mt(nR,-1),vis(nL,0){}
    void add_edge(int u,int v){ g[u].push_back(v); }
    bool dfs(int u){
        if(vis[u]) return false; vis[u]=1;
        for(int v:g[u]){
            if(mt[v]==-1 || dfs(mt[v])){ mt[v]=u; return true; }
        }
        return false;
    }
    int max_matching(){
        int match=0;
        for(int u=0;u<nL;u++){ fill(vis.begin(), vis.end(), 0); if(dfs(u)) match++; }
        return match;
    }
};
```

Dinic (Flujo Máximo)

```
struct Dinic {
    struct E{ int to; long long cap; int rev; };
    int n; vector<vector<E>> g; vector<int> level, it;
    Dinic(int n):n(n),g(n),level(n),it(n){}
    void add_edge(int u,int v,long long c){
        E a{v,c,(int)g[v].size()}, b{u,0,(int)g[u].size()};
        g[u].push_back(a); g[v].push_back(b);
    }
    bool bfs(int s,int t){
        fill(level.begin(), level.end(), -1);
        queue<int> q; q.push(s); level[s]=0;
        while(!q.empty()){
            int u=q.front(); q.pop();
            for(auto &e:g[u]) if(e.cap>0 && level[e.to]==-1){
                level[e.to]=level[u]+1; q.push(e.to);
            }
        }
        return level[t]!=-1;
    }
    long long dfs(int u,int t,long long f){
        if(u==t) return f;
        for(int &i=it[u]; i<(int)g[u].size(); i++){
            auto &e=g[u][i];
            if(e.cap>0 && level[e.to]==level[u]+1){
                long long ret=dfs(e.to,t,min(f,e.cap));
                if(ret){
                    e.cap-=ret;
                    g[e.to][e.rev].cap+=ret;
                    return ret;
                }
            }
        }
        return 0;
    }
    long long maxflow(int s,int t){
        long long flow=0, aug;
        while(bfs(s,t)){
            fill(it.begin(), it.end(), 0);
            while((aug=dfs(s,t,LLONG_MAX))) flow+=aug;
        }
        return flow;
    }
};
```

Bloque 4: Programación dinámica y geometría

Bitmask DP (TSP básico)

```
// dist[i][j] dada. Devuelve costo mínimo para visitar todos desde 0 y volver a 0.
long long tsp(const vector<vector<long long>>& dist){
    int n=dist.size();
    int N=1<<n;
    const long long INF = (1LL<<60);
    vector<vector<long long>> dp(N, vector<long long>(n, INF));
    dp[1][0]=0;
    for(int mask=1; mask<N; mask++){
        for(int u=0; u<n; u++) if(dp[mask][u]<INF){
            for(int v=0; v<n; v++) if(!(mask&(1<<v))){
                dp[mask|1<<v][v]=min(dp[mask|1<<v][v], dp[mask][u]+dist[u][v]);
            }
        }
    }
    long long ans=INF;
    for(int u=0;u<n;u++) ans=min(ans, dp[N-1][u]+dist[u][0]);
    return ans;
}
```

Digit DP (conteo de números <= N con restricción)

```
// Ejemplo: contar números en [0,N] sin dígito repetido (plantilla generalizable).
long long digit_dp_no_repeat(string s){
    int n=s.size();
    static long long memo[20][2][1<<10]; // pos, tight, used mask
    memset(memo,-1,sizeof(memo));
    function<long long(int,int,int,int)> f = [&](int pos,int tight,int started,int mask)->long
long{
        if(pos==n) return started?1:1; // cuenta el 0 también
        long long &res=memo[pos][tight][mask];
        if(started && res!=-1 && !tight) return res;
        long long ans=0;
        int lim = tight? (s[pos]-'0') : 9;
        for(int d=0; d<=lim; d++){
            int nt = (tight && d==lim);
            if(!started && d==0){
                ans += f(pos+1, nt, 0, mask);
            }else{
                if(mask&(1<<d)) continue;
                ans += f(pos+1, nt, 1, mask|(1<<d));
            }
        }
        if(started && !tight) res=ans;
        return ans;
    };
    return f(0,1,0,0);
}
```

Geometría: Punto, orientación, intersección de segmentos

```
// Usar long double si se requiere mayor precisión.
struct P {
    long double x,y;
    P(){} P(long double x,long double y):x(x),y(y){}
    P operator+(const P& o)const{ return P(x+o.x, y+o.y); }
    P operator-(const P& o)const{ return P(x-o.x, y-o.y); }
    P operator*(long double k)const{ return P(x*k, y*k); }
};
long double cross(const P& a,const P& b){ return a.x*b.y - a.y*b.x; }
long double dot(const P& a,const P& b){ return a.x*b.x + a.y*b.y; }
int sgn(long double v){ const long double EPS=1e-12; return (v>EPS)-(v<-EPS); }

int orient(const P& a,const P& b,const P& c){ return sgn(cross(b-a, c-a)); }

bool on_segment(const P& a,const P& b,const P& p){
    if(orient(a,b,p)!=0) return false;
    return (min(a.x,b.x)-1e-12<=p.x && p.x<=max(a.x,b.x)+1e-12 &&
            min(a.y,b.y)-1e-12<=p.y && p.y<=max(a.y,b.y)+1e-12);
}

bool seg_intersects(const P& a,const P& b,const P& c,const P& d){
    int o1=orient(a,b,c), o2=orient(a,b,d);
    int o3=orient(c,d,a), o4=orient(c,d,b);
    if(o1*o2<0 && o3*o4<0) return true;
    if(o1==0 && on_segment(a,b,c)) return true;
    if(o2==0 && on_segment(a,b,d)) return true;
    if(o3==0 && on_segment(c,d,a)) return true;
    if(o4==0 && on_segment(c,d,b)) return true;
    return false;
}
```

DP sobre árboles (tamaños de subárbol, ejemplo)

```
// Calcula tamaño de subárbol y suma de distancias desde root.
struct TreeDP {
    int n; vector<vector<int>> g; vector<int> sz; vector<long long> down;
    TreeDP(int n):n(n),g(n),sz(n,0),down(n,0){}
    void add_edge(int u,int v){ g[u].push_back(v); g[v].push_back(u); }
    void dfs(int u,int p){
        sz[u]=1; down[u]=0;
        for(int v:g[u]) if(v!=p){
            dfs(v,u);
            sz[u]+=sz[v];
            down[u]+=down[v]+sz[v];
        }
    }
};
```

Convex Hull (Monotone Chain) + Área de polígono

```
vector<P> convex_hull(vector<P> v){
    sort(v.begin(), v.end(), [](const P& a,const P& b){
        if(a.x==b.x) return a.y<b.y; return a.x<b.x;
    });
    vector<P> lo, up;
    for(auto p:v){
        while(lo.size()>=2 && orient(lo[lo.size()-2], lo.back(), p)<=0) lo.pop_back();
        lo.push_back(p);
    }
    reverse(v.begin(), v.end());
    for(auto p:v){
        while(up.size()>=2 && orient(up[up.size()-2], up.back(), p)<=0) up.pop_back();
        up.push_back(p);
    }
    lo.pop_back(); up.pop_back();
    lo.insert(lo.end(), up.begin(), up.end());
    return lo; // antihorario
}
long double polygon_area(const vector<P>& poly){
    long double s=0; int n=poly.size();
    for(int i=0;i<n;i++) s += cross(poly[i], poly[(i+1)%n]);
    return fabsl(s)/2.0L;
}
```

Ternary Search (unimodal en reales)

```
// Minimiza f en [l,r] unimodal. Iteraciones suficientes para dobles.
template<typename F>
long double ternary_search_real(long double l,long double r,F f){
    for(int it=0; it<200; it++){
        long double m1 = l + (r-l)/3.0L;
        long double m2 = r - (r-l)/3.0L;
        if(f(m1) < f(m2)) r=m2; else l=m1;
    }
    return (l+r)/2.0L;
}
```

Binary Search en la respuesta (patrón)

```
// Busca el mínimo x en [lo, hi] que satisface check(x).
template<typename F>
long long binary_search_answer(long long lo,long long hi,F check){
    while(lo<hi){
        long long mid = lo + (hi-lo)/2;
        if(check(mid)) hi=mid; else lo=mid+1;
    }
    return lo;
}
```

Utilidades de bits

```
// popcount, lowbit, iterar subconjuntos de una máscara.
int pc(unsigned int x){ return __builtin_popcount(x); }
int pcll(unsigned long long x){ return __builtin_popcountll(x); }
int lowbit(int x){ return x & -x; }

vector<int> subsets(int mask){
    vector<int> v;
    for(int s=mask; s; s=(s-1)&mask) v.push_back(s);
    v.push_back(0);
    return v;
}
```

Mochila completa (unbounded knapsack)

```
// Cada ítem puede usarse infinitas veces.
// dp[w] = valor máximo con capacidad exacta w. O(n*W)
long long knap_unbounded(int W, const vector<int>& wt, const vector<int>& val){
    vector<long long> dp(W+1, 0);
    int n=wt.size();
    for(int i=0;i<n;i++){
        for(int w=wt[i]; w<=W; w++){
            dp[w] = max(dp[w], dp[w-wt[i]] + val[i]);
        }
    }
    return dp[W];
}
```

LIS y LDS O(n log n)

```
// Longest Increasing Subsequence: estrictamente creciente (length).
int LIS_len(const vector<int>& a){
    vector<int> d;
    for(int x : a){
        auto it = lower_bound(d.begin(), d.end(), x);
        if(it==d.end()) d.push_back(x);
        else *it = x;
    }
    return (int)d.size();
}
// Longest Decreasing Subsequence: aplicar LIS a -a[i] o usar upper_bound invertido.
int LDS_len(const vector<int>& a){
    vector<int> d;
    for(int x : a){
        auto it = lower_bound(d.begin(), d.end(), -x);
        if(it==d.end()) d.push_back(-x);
        else *it = -x;
    }
    return (int)d.size();
}
```

Bitmask DP

```
// Ejemplo: dp[mask] = mínimo costo para cubrir conjunto 'mask' desde base.
// Inicializa dp con INF salvo dp[0]=0 y transiciona con opciones precalculadas cost[add][mask]
const long long INF64 = (1LL<<60);
vector<long long> bitmask_dp(int n, const vector<long long>& cost_add){
    // cost_add[x] = costo de activar el bit 'x' en cualquier estado (ejemplo simple).
    int N = 1<<n;
    vector<long long> dp(N, INF64);
    dp[0]=0;
    for(int mask=0; mask<N; mask++){
        long long cur = dp[mask];
        if(cur>=INF64) continue;
        for(int b=0;b<n;b++) if(!(mask&(1<<b))){
            dp[mask | (1<<b)] = min(dp[mask | (1<<b)], cur + cost_add[b]);
        }
    }
    return dp;
}
```

Digit DP

```
// Cuenta números en [0..N] cuya suma de dígitos ≡ S (mod M). N dado como string.
long long digit_dp_sum_mod(string N, int M, int S){
    int n=N.size();
    static long long memo[20][2][200]; // pos,tight,sum_mod (M<=200 aprox)
    memset(memo, -1, sizeof(memo));
    function<long long(int,int,int)> f = [&](int pos,int tight,int sum)->long long{
        if(pos==n) return (sum%M+M)%M==S;
        long long &res = memo[pos][tight][(sum%M+M)%M];
        if(res!=-1) return res;
        res=0;
        int lim = tight ? (N[pos]-'0') : 9;
        for(int d=0; d<=lim; d++){
            res += f(pos+1, tight && (d==lim), (sum+d)%M);
        }
        return res;
    };
    return f(0,1,0);
}
```

DP sobre árboles

```
// Calcula: sumDist[u] = suma de distancias desde u a todos los nodos.
// Paso 1: dfs1 para tamaños y sumDist[root]; Paso 2: dfs2 para re-enraizar.
struct TreeReroot {
    int n; vector<vector<int>> g;
    vector<int> sz; vector<long long> dp; // dp[u] = sum dist desde root (parcial)
    vector<long long> ans;               // ans[u] = sum dist desde u
    TreeReroot(int n):n(n),g(n),sz(n,0),dp(n,0),ans(n,0){}
    void add_edge(int u,int v){ g[u].push_back(v); g[v].push_back(u); }
    void dfs1(int u,int p){
        sz[u]=1; dp[u]=0;
        for(int v:g[u]) if(v!=p){
            dfs1(v,u);
            sz[u]+=sz[v];
            dp[u]+=dp[v]+sz[v];
        }
    }
    void dfs2(int u,int p){
        ans[u]=dp[u];
        for(int v:g[u]) if(v!=p){
            long long pu_dp=dp[u], pv_dp=dp[v];
            int pu_sz=sz[u], pv_sz=sz[v];

            // quitar contribución de v en u
            dp[u] -= dp[v] + sz[v];
            sz[u] -= sz[v];
            // añadir contribución de u en v
            dp[v] += dp[u] + sz[u];
            sz[v] += sz[u];

            dfs2(v,u);

            // revertir
            dp[u]=pu_dp; dp[v]=pv_dp; sz[u]=pu_sz; sz[v]=pv_sz;
        }
    }
    vector<long long> run(int root=0){ dfs1(root,-1); dfs2(root,-1); return ans; }
};
```

DP con divide & conquer optimization

```
// Optimiza una transición dp[k][i] = min_{j<i} ( dp[k-1][j] + C(j,i) ) asumiendo que el argmin
es monótono.
// Requiere que la función C cumpla condiciones de cuadrángulo (divide & conquer optimization).
// L: índice izquierdo, R: derecho; optL..optR: límites del mejor j.
template<typename Cost>
void compute_dp_layer(int L,int R,int optL,int optR,
                      const vector<long long>& dp_prev, vector<long long>& dp_cur, Cost C){
    if(L>R) return;
    int mid = (L+R)/2;
    pair<long long,int> best = {LLONG_MAX, -1};
    int start = optL, end = min(mid-1, optR);
    for(int j=start; j<=end; j++){
        long long v = dp_prev[j] + C(j, mid);
        if(v < best.first){ best = {v, j}; }
    }
    dp_cur[mid] = best.first;
    int opt = (best.second==-1 ? start : best.second);
    compute_dp_layer(L, mid-1, optL, opt, dp_prev, dp_cur, C);
    compute_dp_layer(mid+1, R, opt, optR, dp_prev, dp_cur, C);
}
```

Cross product y orientación

```
inline long double cross2(long double ax,long double ay,long double bx,long double by){ return
ax*by - ay*bx; }
int orientation_ll(const Point& a,const Point& b,const Point& c){
    long double v = cross(b-a, c-a);
    return (v>1e-12) - (v<-1e-12);
}
```

Intersección de segmentos

```
// Asume que se sabe que las rectas (no segmentos) se intersecan en un punto (no paralelas).
bool line_intersection(Line L1, Line L2, Point& out){
    long double d = cross(L1.v, L2.v);
    if(fabsl(d) < 1e-15) return false; // paralelas
    Point u = L2.p - L1.p;
    long double t = cross(u, L2.v) / d;
    out = L1.p + L1.v * t;
    return true;
}
```

Estructuras Point, Line, Segment

```
struct Point {
    long double x,y;
    Point(){} Point(long double x,long double y):x(x),y(y){}
    Point operator+(const Point& o)const{ return {x+o.x,y+o.y}; }
    Point operator-(const Point& o)const{ return {x-o.x,y-o.y}; }
    Point operator*(long double k)const{ return {x*k,y*k}; }
};
inline long double dot(const Point& a,const Point& b){ return a.x*b.x + a.y*b.y; }
inline long double cross(const Point& a,const Point& b){ return a.x*b.y - a.y*b.x; }
inline int sgn(long double v){ const long double EPS=1e-12; return (v>EPS)-(v<-EPS); }

struct Line {
    // Representación: punto p y dirección v (no nula): p + t*v
    Point p, v;
    Line(){} Line(Point p, Point v):p(p),v(v){}
};

struct Segment {
    Point a,b;
    Segment(){} Segment(Point a,Point b):a(a),b(b){}
};

int orient(const Point& a,const Point& b,const Point& c){ return sgn(cross(b-a, c-a)); }

bool on_segment(const Point& a,const Point& b,const Point& p){
    if(orient(a,b,p)!=0) return false;
    return (min(a.x,b.x)-1e-12<=p.x && p.x<=max(a.x,b.x)+1e-12 &&
            min(a.y,b.y)-1e-12<=p.y && p.y<=max(a.y,b.y)+1e-12);
}

bool seg_intersects(const Segment& s1, const Segment& s2){
    const Point &a=s1.a,&b=s1.b,&c=s2.a,&d=s2.b;
    int o1=orient(a,b,c), o2=orient(a,b,d);
    int o3=orient(c,d,a), o4=orient(c,d,b);
    if(o1*o2<0 && o3*o4<0) return true;
    if(o1==0 && on_segment(a,b,c)) return true;
    if(o2==0 && on_segment(a,b,d)) return true;
    if(o3==0 && on_segment(c,d,a)) return true;
    if(o4==0 && on_segment(c,d,b)) return true;
    return false;
}

// Distancia punto-recta (línea infinita)
long double dist_point_line(Point p, Line L){
    // |(p - L.p) x L.v| / |L.v|
    long double num = fabsl(cross(p - L.p, L.v));
    long double den = hypot(L.v.x, L.v.y);
    return num / den;
}
```

Polygon area (Shoelace)

```
long double polygon_area(const vector<Point>& poly){
    long double s=0; int n=poly.size();
    for(int i=0;i<n;i++){
        s += cross(poly[i], poly[(i+1)%n]);
    }
    return fabsl(s)/2.0L;
}
```